

Quality Assurance for Microservice Architectures

Thomas Schirgi

*Institute of Technical Informatics
Technical University Graz
Graz, Austria
thomas.schirgi@student.tugraz.at*

Eugen Brenner

*Institute of Technical Informatics
Technical University Graz
Graz, Austria
brenner@tugraz.at*

Abstract—Microservices are becoming more and more popular. Appropriate quality assurance for Microservices is necessary since the user experience for a system is also directly linked to performance and efficiency. This paper gives an overview of quality assurance for Microservices Architecture and shows some Architectural Smells and Anti-pattern.

Microservice Architecture; Anti-pattern; Architectural Smells; Quality Assurance

I. INTRODUCTION

Since Microservices was first mentioned by Fowler and Lewis in 2014 [1], this type of architecture seems to be a paradigm change in software development. This can be seen in the Hype Cycle for Application and Integration Infrastructure published by Gartner in 2019 [2]. Many IT Companies – like the FANG companies – deliver their services based on Microservices. Contrary to that, a widely used architecture is a monolith, where all functionality of an application has to be deployed together.

II. SOFTWARE QUALITY

In the IEC 90003:2018, a software product is defined as a "set of computer programs, procedures and possibility associated documentation and data" [3]. The source code quality of such a product is a significant part of the entire quality of an application. Issues in the source code of an application can lead to problems ranking from exceptions to application crashes or even to damages to the system. Furthermore, such cases may cause monetary loss and a loss of reputation. According to [4], it must be distinguished between faults and failures.

- **Faults:** a software fault is a software error that causes improper functioning of the Software. In some software fault cases, the fault is corrected or can be neutralized by following code lines. E.g., Exception Handler
- **Failures:** software failures disrupt the use of Software and are result from a software fault. E.g., Software closes.

To ensure that there are no faults and failures in a software product, developers need to provide good code quality and architectural quality. In [5], the importance of software architecture is summarized in six aspects:

1. **Understanding:** Software architecture is a mechanism to simplify the ability to understand complex and large systems by representing them at a higher level of abstraction.
2. **Reuse:** It supports the reuse of components and frameworks. There are different promoters for reuse, like architectural patterns, domain-specific architecture, components, etc.
3. **Construction:** It provided some blueprint for development and implementation by showing the major components and dependencies between them.
4. **Evolution:** Software architecture shows possibilities where a system may evolve in the future.
5. **Analysis:** It can be seen as a mechanism to analyze the entire system. These analyzes may also include consistency checking, conformance to constraints, etc.
6. **Management:** Software architecture can be seen as a huge aspect of any software development process. Evaluation of architecture leads to a clear understanding of requirements, implementation plans, and possible risks. This will reduce the amount of rework that needs to be done to address problems later in the system's lifetime.

A. Quality Attributes

A quality attribute (QA) is a non-functional requirement and a measurable feature of a system used to determine how well a system satisfies the stakeholders. In the ISO/IEC 25010 – also known as SQuaRE (Software product **Q**uality **R**epresentations and **E**valuation) – the product quality properties are categorized into eight characteristics [6]: Functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability and portability. Each of these characteristics is itself composed of a set of related sub-characteristics.

B. Software Quality Measurement

Measurement in software engineering is seen as a crucial factor in evaluating quality characteristics like functionality, usability, reliability, etc. Measurement allows to understand the current situation and helps to develop clear benchmarks that are useful to set goals for future behavior. Whereas some measurement principles are becoming more and more over-

stressed, other equally essential design methods have been omitted in the architecture measurement process.

According to [3], the main objectives of software quality measurement are:

- To assist the management in Monitoring and controlling the development and maintenance of software systems
- Observing the conformance of Software to function and other requirements
- Serving as a data source for improvement by identifying low performance and demonstrating the achievements of corrective actions

Some of the quality attributes can be measured using various software architecture characteristics like size, complexity, coupling, cohesion, etc. Those measures are well defined in the IEC 25023 [7]. Furthermore, each quality attribute can be measured by combining it with others.

In [6], it is defined that the measurement is a logical sequence of operation to quantify the properties concerning a specific scale. The quality characteristics and their sub characteristics can be quantified by applying measurement functions or an algorithm used to combine quality measure elements. More than one software quality measure may be used to measure a quality characteristic or its uncharacteristic. An example for this is stated in [8], where multiple metrics are combined to measure a specific property using either composition or aggregation:

- Composition
 - A simple or weighted average of the metrics can only be used when the different metrics use the same ranges and semantics.
 - Thresholding
 - Interpolation
 - Additionally, a combination of the values above can also be used.
- Aggregation (several steps are required)
 1. Apply a weighting function to each metric
 2. Average the weighted values
 3. Compute the inverse function of the average

III. QUALITY ATTRIBUTES FOR MICROSERVICE ARCHITECTURE

According to [9], there is no relatively systematic panorama for state-of-the-art quality attributes for Microservice Architecture (MSA) yet. It is still unknown how QAs are impacted in MSA and which QAs are the most challenging ones. Hence many aspects are still unclear, unexplored or even inconsistent. Based on [9], the most concerned QAs for Microservices are scalability, performance, availability, monitorability, security and testability.

A. Scalability

Scalability can be done in three dimensions: horizontally (x-axis), vertically (y-axis) and lookup oriented (z-axis). When scaling horizontally (scaling out), more resources to logical units are added – like servers to an already existing cluster. When scaling vertically (scaling-up), more resources to a physical unit are added (like memory to a single computer). With Z-Axis scaling, each server runs an identical copy of the code, similar to X-Axis scaling. The difference is that each server is only responsible for a part of the data. A component of the server is responsible for routing requests to the appropriate server. A possible routing criteria would be the primary key of a requested entity. This splitting method is used to scale databases, where data is partitioned (or sharded) over a set of servers based on an attribute of the dataset.

B. Performance

Performance is measuring a system's ability to meet specific timing requirements to respond to an event. The allocation of different amounts of resources may influence the throughput or the response time of service. On the other side, the communication among Microservices via the network may directly impact the performance of systems. Thus performance and scalability can rely on each other: Smaller Microservice may increase the scalability but may decrease the performance due to a higher amount of interactions between the services. In contrast, the performance may be improved if two Microservices are merged to reduce the communication overhead.

C. Availability

Availability measures the ability of a system to repair faults so that the period of an unavailable service does not exceed a required value [10]. The availability of a system can be calculated as the probability that it will provide the specified services within required bounds over a time interval.

D. Monitorability

Monitorability is used to measure the operations staff's ability to supervise the system while it is executing. It is a complex part of Microservice-based systems but becomes more and more essential due to the high level of dynamic structure and behavior of such a system. As an example, the Monitoring might be on infrastructure level (VM or Container), application level (e.g., response time), or on environment level (e.g., network). In addition, improving monitorability may also impact other QAs like scalability, performance and availability [10].

E. Security

Security measures the system's ability to protect data and information against unauthorized access while still providing access to people and systems that are authorized. Since an MSA distributes logic among multiple services, the network interaction is getting more complex. Thus attackers can exploit this complexity to launch specific attacks.

F. Testability

As the relationship between Microservices might be complex and some Microservices are evolving over time, it is

necessary to pay attention to the testability of Microservices. According to [11], software testing is defined as “An activity in which a system or component is executed under special conditions, the results are observed or recorded and evaluation is made of some aspect of the system or the component.” Testability directly impacts performance, availability, and security and is a measure of a system's ability to demonstrate its faults through (typically execution-based) testing.

- Unit Test: Typically, Functions and Methods are tested. No services are started and the usage of external files and network connections will be limited (Classes within Service)
- Integration Test: Tests Inbound and Outbound Adapters and their dependencies. (Service Adapters)
- Component Test: Testing is performed on each component separately without integrating with other Components (Entire Service)
- End-to-End Test: End-to-end tests involve the entire system. Sometimes also the user interface comes into account. (Multiple Services or Application)

IV. MICROSERVICE ARCHITECTURE BEST PRACTICES

To create a sustainable architecture for Microservices, some principles must be met [12]:

- A Microservice Architecture must split its item into modules.
- Modules should interact via non-proprietary interfaces
- Specific modules should not depend on implementation details of other modules
- Integration options must be limited and standardized
- Communication should be limited to RESTful Web-services or Messaging Protocols.
- Authentication must be standardized on all services.
- Each Module should have a continuous delivery pipeline
- The operation should be standardized: Same configuration, deployment, log analysis, etc.
- Modules should be resilient. Modules should not fail if other modules are not reachable.
- Two independent layers in terms of architectural approach:
 - Macro Architecture: Decisions regarding all modules (e.g., Authentication)
 - Micro Architecture: Decisions that can be made for every single Module. (e.g., Database System)

But if all of those principles are met, it is not sure that the architecture is well designed and without mistakes. The complexity of an MSA leads to attack surfaces in terms of bad decisions when defining and implementing the architecture.

Therefore it can be distinguished between Anti-pattern and Code-Smells. Sometimes Anti-pattern and Smells are seen as the same issues. Also, the transition in literature is unclear and sometimes blurred. To distinguish, Architectural Smells and Anti-pattern can be defined as followed:

- Architectural Smells: Smells should be investigated. Smells may and may not be bad.
- Anti-pattern: This will lead to a worse design of a system. Anti-patterns are always bad and should be solved immediately.

In the following sections, some Architectural Smells and Anti-pattern concerning Microservices are discussed. It was tried to distinguish between Bad Smells and Anti-pattern, but – as mentioned – this difference is blurred.

A. Architectural Smells

In this part, some Architectural Smells within MSA are discussed. It is also discussed how such smells can be prevented and/or solved.

1) Hard-Coded Endpoints

IP Addresses and ports are hard-coded of the service which need to be used. This leads to problems when the service location needs to be changed [13]. This could be solved by using either a service discovery or a well-designed DNS System with the ports set in a configuration file.

2) Shared Persistence

Different Microservices are accessing the same database. In the worst kind, other services are using the same entities of a service. This approach couples the Microservices connected to the same data and reduces the service independence. Solutions for the smell are [14]

- usage of independent databases for each service,
- use of a shared database with a set of private tables for each service which the service can only access, or
- usage of a private database schema for each service

3) Independent Deployability

In MSA, each service should be running independently from another service, so it should be possible to deploy and un-deploy a Microservice independently from others [15]. This also implies that a Microservice can be started without waiting for other Microservices to be running.

Container Systems (like Docker) provide an ideal way to deploy independently if each service runs in its container. If two services (A & B) are placed in the same container, they would operationally depend on each other, since launching a new instance of service A would also start an instance of service B.

4) Horizontal Scalability

A consequence of independent deployability is the possibility of adding and removing replicas of a Microservice. To make sure that the architecture is horizontal scaleable, all replicas of Microservice A should be reachable by the Microservices invoking A.

This smell occurs when no API Gateway is used or locations of other services are hard-coded into the source code (Endpoint-Based service Interactions). To solve the smell of Endpoint-Based service interactions, a service registry should be implemented [16].

5) *Isolation of failures*

A Microservice can fail for many reasons: Hardware issues, network issues, application-level issues, bugs, etc., and becomes unavailable for other Microservices. Furthermore, the communication may fail due to the high complexity of the entire system. Therefore Microservices should be designed to tolerate such failures and remain up and running. Failures should not be cascaded to underlying services, which means that a failure in Microservice A results in triggering a failure in Microservice B. The most common solution is to use a Circuit Breaker [17].

6) *Decentralization*

In all aspects of Microservice-based Applications, decentralization should occur, implying that the business logic should be fully decentralized and distributed among the Microservices [18]. This happens whenever an Enterprise Service Bus is misused. Whenever an ESB is placed as a central hub – with the other Microservices as spokes – it becomes a bottleneck. This approach may lead to undesired centralization of business logic [19].

B. *Anti-Pattern*

Anti-Pattern are leading to a worse design of an Architecture. According to [20], Anti-pattern can be divided into four groups. Some examples of Anti-pattern are presented in each group based on [13] and [14] - are presented and ways to solve them are shown.

1) *Design Anti-pattern*

- **Wrong Cut:** Occurs if Microservices are split based on technical layers (presentation, business and data layers) instead of business capabilities, leading to higher complexity. To mitigate this, a precise analysis of business processes needs to be performed.
- **Cyclic Dependencies:** A Service B is called by A, B calls C and C again calls A. Microservices involved in cyclic dependencies can be hard to maintain or reused in isolation. As a solution, the cycle should be refined and an API Gateway should be applied.
- **Nano Service:** Single Microservice should be designed to fulfill single business capabilities, not more but also not less. Nano services may increase the opportunities for reuse and help developers to focus on essential Microservices. Nevertheless, they require more developers and the context is often switching. During run-time, Nano Services are creating more overhead and communication.
- **Mega Service:** As opposite to Nano Services, also Mega services should be avoided. Microservices should be designed to fulfill only one business capability. Mega Services are difficult to test and also create maintenance efforts and increased complexity.

2) *Implementation Anti-pattern*

- **Shared Libraries:** Microservices should not share run-time libraries and source code directly. This somehow breaks the boundaries between Microservices, which cannot be seen as independent and independently deployable.
- **Too many standards:** Although Microservices are allowing different technologies, too many different protocols, frameworks, development languages, etc., are used. This may lead to problems in companies, especially in the event of developer turnovers.
- **Too new technology:** The technology used needs to be defined well. Too new technology is not always the best choice since it might not be fully developed.

Depending on the definition of Anti-pattern, also Hard-Coded Endpoints can be seen as an implementation Anti-pattern.

3) *Deployment Anti-pattern*

- **Manual Anti-pattern:** Microservices are relying on some sort of automation. Everything, which is possible, should be automated. Therefore a configuration file for each Microservice is not the best solution. Instead, a configuration server should be used, which automates the configuration process.
- **No Continuous Integration (CI) / Continuous Delivery (CD):** The independent deployability of Microservices also offers the possibility to apply iterative, continuous development and deployment (DevOps) processes. DevOps' integration results in a reduced delivery time, increased delivery efficiency, a decreased time between releases, and higher software quality.
- **No API Gateway:** Microservices are communicating directly with each other. If no API Gateway is in place, the service consumers are also communicating directly with the Microservices. This increases the complexity of the system but also makes a system harder to maintain.
- **Timeouts:** The availability of a Microservice refers to the possibility for each service consumer to connect and send a request. In distributed systems, consumer applications/tasks use timeouts to handle unavailability or unresponsiveness. It is hard to find the correct timeout value: a too-short value will quickly lead to exceptions, a too-long value will force clients' applications to wait too long before stopping the request.
- **No API Versioning:** In some cases, multiple Versions of APIs must be exposed by a service. This is usually the case if significant changes had been done on a particular Microservice. Therefore a version number can be part of the request URL, or the version can be inserted into the HTTP header of the request.

4) *Monitoring Anti-pattern*

- **No Health Check:** Since a Microservice can be deployed everywhere and can be unavailable due to specific reasons, the status of a Microservice might be unknown. Therefore a health check API endpoint

should be implemented, which periodically verifies the health status and the ability to answer requests.

- **Local Logging:** During run-time, each Microservice produces a lot of information with logging. These logs are usually stored directly on the file system and cannot be accessed directly. Therefore a central logging service should be implemented and used. Distributed logging mechanisms are easy to implement and make debugging more accessible (like Elasticsearch).
- **Insufficient Monitoring:** If Microservices are part of Service Level Agreements (SLA), the behavior and performance are crucial. Insufficient Monitoring is linked to "Local Logging" and may hinder maintenance activities in a Microservice-based system because failures become challenging to catch and performance tracking is unavailable. This can even affect the ability to comply with the SLA. To solve this, a global monitoring tool should be implemented.

V. CONCLUSION

As a more and more accepted and adopted architectural style, MSA overcomes the limitations of traditional monolithic architecture. To avoid Anti-pattern and Architectural Smells, the correct granularity for the Microservices has to be found. Before starting with the implementation, a big-picture, strategy and structure has to be found. Without those, the implementation will undoubtedly lead to anti-pattern and architectural smells.

Furthermore, the best solution must be found for each project. Solutions of big players like Netflix, Facebook and so on sometimes do not fit for small applications and vice versa. Even if Microservice architecture is emerging, it is not always the best choice and needs to be investigated carefully.

REFERENCES

- [1] J. Lewis and M. Fowler, *Microservices - A definition of this new architectural term*, 2020.
- [2] T. Permikangas, *Digital Ecosystem Development: The future of integration environment*, 2020.
- [3] ISO, "ISO/IEC/IEEE 90003:2018 - Software engineering: Guidelines for the application of ISO 9001:2015 to computer software," Genf, 2018.
- [4] D. Galin, *Software Quality Concepts and Practice*, 2018.
- [5] M. Alenezi, "Software Architecture Quality Measurement Stability and Understandability," *International Journal of Advanced Computer Science and Applications*, Bd. 7, pp. 550-559, 8 2016.
- [6] ISO, "ISO/IEC 25010 Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models," 2011.
- [7] ISO, "ISO/IEC 25023 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Measurement of system and software product quality," 2011.
- [8] K. Mordal-Manet, N. Anquetil, J. Laval, A. Serebrenik, B. Vasilescu and S. Ducasse, "Software quality metrics aggregation in industry," *Journal of Software: Evolution and Process*, Bd. 25, pp. 1117-1135, 10 2013.
- [9] S. Li, H. Zhang, Z. Jia, C. Zhong, C. Zhang, Z. Shan, J. Shen and M. A. Babar, "Understanding and addressing quality attributes of microservices architecture: A Systematic literature review," *Information and Software Technology*, Bd. 131, p. 106449, 2021.
- [10] L. Bass, P. Clements, and R. Kazman, *Software architecture in practice*, Addison-Wesley Professional, 2012.
- [11] ISO, "ISO/IEC/IEEE 29119-1:2013 Software and systems engineering - Software testing - Part 1: Concepts and definitions," *ISO / IEC / IEEE*, p. 56, 2013.
- [12] W. Eberhard, *The Microservice Practice Book - Das Microservices-Praxisbuch: Grundlagen, Konzepte und Rezepte*, dpunkt.verlag, 2018.
- [13] I. Pigazzini, F. A. Fontana, V. Lenarduzzi und D. Taibi, "Towards Microservice Smells Detection," in *Proceedings of the 3rd International Conference on Technical Debt*, New York, NY, USA, 2020.
- [14] D. Taibi und V. Lenarduzzi, "On the Definition of Microservice Bad Smells," *IEEE Software*, Bd. vol 35, 5 2018.
- [15] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*, O'Reilly, 2019.
- [16] M. L. Abbott, *The art of scalability : : scalable web architecture, processes, and organizations for the modern enterprise*, Second edition.. ed., 2015.
- [17] K. Torkura, M. I. H. Sukmana und C. Meinel, "Integrating Continuous Security Assessments in Microservices and Cloud-Native Applications," 2017.
- [18] O. Zimmermann, "Microservices tenets," Bd. 32, Springer Science and Business Media LLC, 2016, p. 301-310.
- [19] M. Rusek, G. Dwornicki und A. Orlowski, "A Decentralized System for Load Balancing of Containerized Microservices in the Cloud," 2017.
- [20] R. Tighilt, M. Abdellatif, N. Moha, H. Mili, G. El-Boussaidi, J. Privat und Y.-G. Guéhéneuc, „On the Study of Microservices Antipatterns: a Catalog Proposal,“ 2020.